

Jan 28th HW 12

2. Explain TLS, PKI, certificate, public key, private key, and signature.

- A public key and private key are a mathematically related key pair.
- Public Key:
 - Can be shared with anyone
 - Used to encrypt data or verify digital signatures
- Private Key
 - Must be kept secret by the owner
 - Used to decrypt data or create digital signatures
- A digital signature is used to guarantee:
 - Authentication (who sent the data)
 - Integrity (data was not changed)
 - Non-repudiation (sender cannot deny sending it)
- A certificate contains:
 - Owner identity (domain name, organization, etc.)
 - Public key
 - Validity period
 - CA's digital signature:
- PKI is the entire trust system that manages public-key security.
 - Certificate Authorities (CA)
 - Digital certificates
 - Public/private key pairs
 - Certificate validation and revocation
- TLS is a security protocol that provides secure communication over a network (used by HTTPS).
 - Encryption (confidentiality)
 - Authentication (server identity)
 - Integrity (data protection)

4. list all http status codes that related to authentication and authorization failures.

- 401 = not authenticated
- 403 = authenticated but not authorized

5. Compare authentication and authorization? Name and explain important components in Spring security that undertake authentication and authorization

- Authentication = Who are you?
 - Verifies identity (username/password, OAuth login, JWT, etc.)
 - Output is an authenticated Authentication object (principal + authorities)
- Authorization = What can you do?
 - Checks permissions/roles against a resource/action
 - Happens after authentication (usually)
- Important Spring Security components (authn/authz):

- Security Filter Chain (SecurityFilterChain)
- AuthenticationFilter (e.g., UsernamePasswordAuthenticationFilter, BearerTokenAuthenticationFilter)
- AuthenticationManager
- AuthenticationProvider
- UserDetailsService
- PasswordEncoder
- Authorization layer
- SecurityContextHolder + SecurityContext

6. Explain HTTP Session?

- An HTTP Session is server-side storage that lets a web app remember a user across multiple requests.
 - HTTP is stateless, so session adds state.
 - Server keeps a session object (key-value data).
 - Client holds only a session id (usually in a cookie like JSESSIONID) that points to the server session.

7. Explain Cookie?

- A cookie is a small piece of data stored in the browser and sent with each request to the same domain/path rules.
- Common uses:
 - Session id (JSESSIONID)
 - “Remember me” token
 - User preferences (language, theme)

8. Compare Session and Cookie?

- Session: Stored in server, size is large, scale is harder
- Cookie: Stored in client, size is small, scale is easier

10. How do we use session and cookie to keep user information across the the application?

- Common pattern (session-based login)
 - User logs in
 - Server creates session + stores user data (or user id)
 - Server sends cookie JSESSIONID=<id>
 - Browser sends cookie on every request
 - Server loads session and knows who the user is

11. What is the spring security filter?

- A Spring Security filter is a servlet filter inside the Security Filter Chain.
- It intercepts HTTP requests before they hit controllers and performs tasks like:
 - read credentials (cookie/token)
 - authenticate user
 - authorize access rules
 - handle security exceptions (401/403)

- CSRF protection, session management, logout, etc.

12. Explain bearer token and how JWT works.

- Bearer token: Whoever bears (possesses) this token is treated as the user.
- Sent in header: Authorization: Bearer <token>
- JWT is a common bearer token format:
 - Header (alg, typ)
 - Payload (claims: user id, roles, exp)
 - Signature (prevents tampering)
- JWT flow:
 - Server issues JWT after login
 - Client stores it (memory / secure cookie)
 - Client sends it each request
 - Server verifies:
 - signature valid
 - token not expired (exp)
 - claims authorize the request
-  JWT signature provides integrity, not encryption (payload is readable unless you use JWE).

13. Explain how do we store sensitive user information such as password and credit card number in DB?

- Passwords:
 - Never store plaintext
 - Store one-way hash using strong algorithm:
 - BCrypt / Argon2 / scrypt (preferred)
 - Also:
 - unique salt (BCrypt includes salt)
 - optional “pepper” in app config (extra secret)
- Credit card numbers:
 - Best practice: don’t store full card numbers at all.
 - Use a payment provider (Stripe, etc.) and store only:
 - token / customer id
 - last4, brand, expiration (non-sensitive)

14. Compare UserDetailsService, AuthenticationProvider, AuthenticationManager, AuthenticationFilter?(把这几个名字看熟悉也行)

- AuthenticationFilter
 - Where: in filter chain (request entry point)
 - Does: extracts credentials from request and calls manager
- AuthenticationManager
 - Does: coordinates authentication, delegates to providers
- AuthenticationProvider

- Does: performs actual credential verification
- Example: checks password or validates JWT
- UserDetailsService
 - Does: loads user record (username/password/roles) from DB
 - Usually used by DaoAuthenticationProvider

15. What is the disadvantage of Session? how to overcome the disadvantage?

- Disadvantages
 - Server state: consumes memory / storage
 - Scaling: hard in distributed systems (multiple servers)
 - Needs sticky sessions or shared session store
 - Session replication can be complex/slow
- Solutions
 - Stateless auth with JWT (no server session)
 - Or store sessions in shared storage:
 - Redis (Spring Session + Redis)
 - Database-backed sessions
 - Load balancer with sticky sessions (works but less ideal)

16. how to get value from application.properties in Spring security?

- Common ways:
- @Value


```
@Value("${jwt.secret}")
private String jwtSecret;
```
- @ConfigurationProperties (cleaner for many values)


```
@ConfigurationProperties(prefix="jwt")
public class JwtProps {
    private String secret;
    private long expirationMs;
    // getters/setters
}
```

17. What is the role of configure(HttpSecurity http) and configure(AuthenticationManagerBuilder auth)?

- configure(HttpSecurity http), Defines authorization rules + web security config:
 - which endpoints are public/protected
 - login/logout
 - CSRF
 - session management
 - add custom filters
 - exception handling (401/403)
- configure(AuthenticationManagerBuilder auth), Defines authentication mechanism:

- user source (in-memory, JDBC, UserDetailsService)
- password encoder
- authentication providers

19. Explain best practices to securely store secrets in applications.

- Never hardcode secrets in code or commit them to Git
- Use a secret manager:
 - AWS Secrets Manager, Azure Key Vault, GCP Secret Manager, HashiCorp Vault
- Use environment variables or external config for local/dev
- Encrypt secrets at rest + restrict access (least privilege IAM)
- Rotate secrets regularly (automation if possible)
- Use separate secrets per environment (dev/test/prod)
- Log redaction: never print tokens/passwords
- For keys:
 - use KMS/HSM when possible
 - separate encryption keys from encrypted data
- For CI/CD:
 - use secured secret storage (GitHub Actions Secrets, etc.)
 - limit who can read/modify them